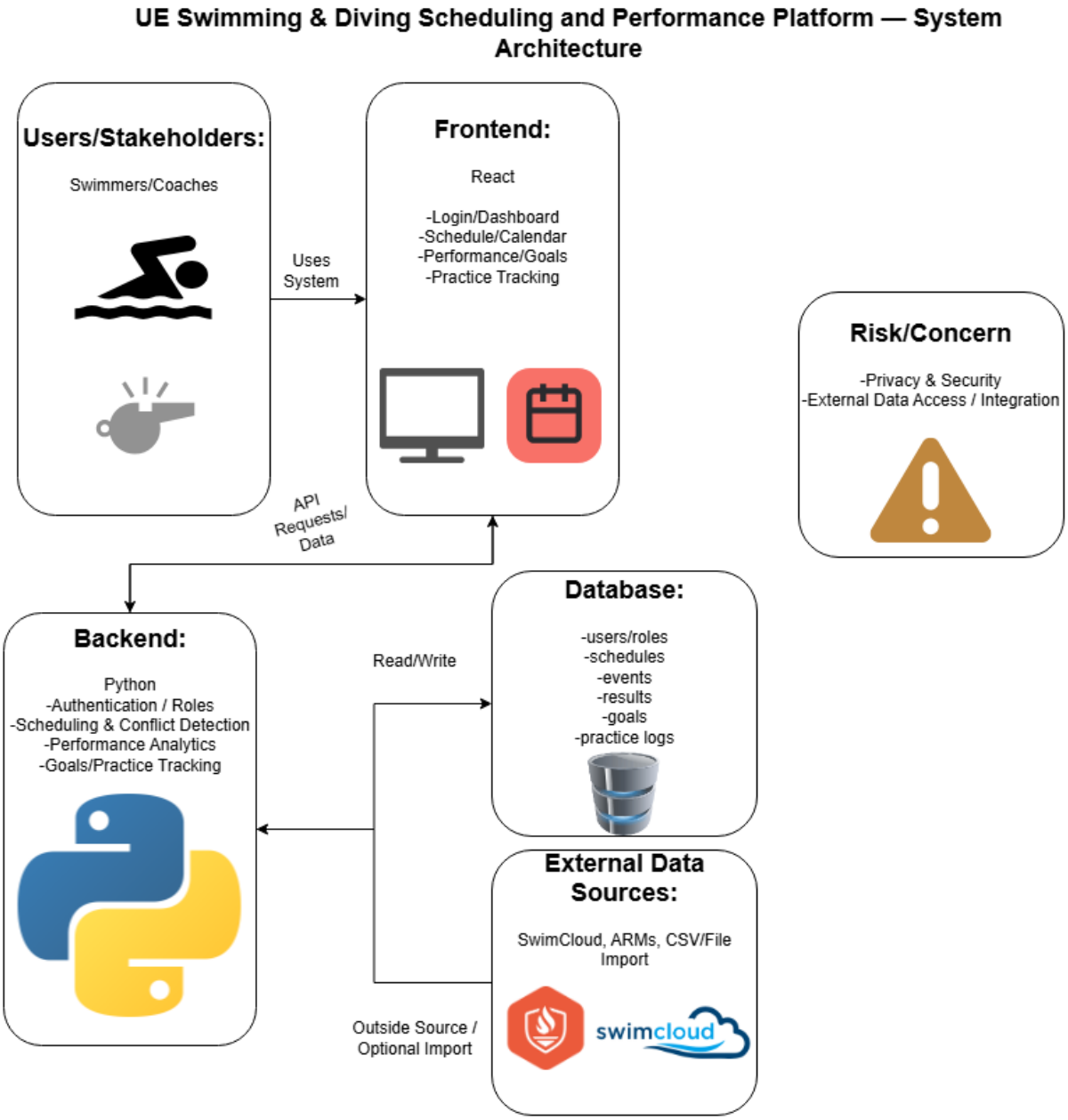
Project Title: Swim Team Scheduling and Performance Platform for University of Evansville Swimming & Diving

Project Overview: The goal of this project is to build a web-based platform for scheduling and performance tailored for the University of Evansville Swimming & Diving team. The primary client for the project is the University of Evansville Swimming & Diving coaching staff, with swimmers also serving as major users and stakeholders. The system will have academic, athletic, and team schedules in one place, while also automatically identifying scheduling conflicts. This will help coaches determine optimal practice times with the fewest conflicts more efficiently as this is done manually by hand currently. This application will provide swimmers and coaches with access to performance information, goals, and other useful team data in one reliable place. This project is intended to reduce the amount of information that is spread across separate systems and make scheduling and performance tracking easier and more efficient for both coaches and swimmers.

System Architecture:



Technology Stack: The frontend for the project will be developed using React with Vite and also JavaScript. I chose to use React after some research because it is suited well for building interactive web interfaces, while Vite gives a simple development environment for the frontend. The backend for the project will use Python with FastAPI and Uvicorn. I chose Python as it is the language I am the most familiar with, and I found that FastAPI provides a very straightforward way to create API endpoints that the frontend can communicate with. I will add a relational database later when persistent data storage is required, with SQLite being considered for the initial prototype because it is lightweight, works locally, and uses SQL, which I am already familiar with. I am using GitHub for version control and project storage, while also using VS Code for development. External systems such as SwimCloud and ARMS may eventually provide the real-world data that I require, but I also may be able to easily get all of that information from my coach directly.

Core Features:

1. Unified Scheduling and Calendar
2. Scheduling Conflict Detection and Low-Conflict Practice-Time Ranking
3. Coach Availability / Scheduling Dashboard
4. Basic Swimming Performance Tracking
5. Coach and Swimmer Accounts with Appropriate Permissions

User Workflow:

Coach setting up practice times for the team: A coach opens the application and views swimmers' class schedules and existing team events. The system processes the schedules and identifies conflicts for possible practice times. It then ranks the available slots for practice that have the lowest number of conflicts. The coach reviews the times that are suggested and selects the options that work best for the team overall. The selected practice time can then be added officially to the team's unified schedule.

Prototype Setup progress: The project now has a private GitHub repository and a local development workspace that has been set up. The project is separated into frontend, backend, and some documentation directories. The frontend has been initialized using React and Vite, while the backend has been initialized with Python with FastAPI and Uvicorn inside a Python virtual environment. The backend and frontend are connected and have been proven to do so using a basic health check. The system architecture, requirements document, and project context are also stored with the project documentation.

Responsible Computing Concerns: The main concerns are privacy and security for this project. The system could potentially contain class schedules, performance data, goals, and other information about student-athletes. I would have to ensure that coaches and athletes have appropriate levels of access to different features and information. I would also have to avoid using performance statistics in ways that create unfair comparisons between athletes.

Feasibility and Risks:

Risk 1: Scope

The project could potentially have many different features, so there is a risk that I attempt to add too much during the development timeline. This is being addressed by prioritizing the core features that are needed to start and postponing the secondary features to a later time.

Risk 2: External Data Access

Data from existing platforms like SwimCloud and ARMS may not be easily accessible. The project will therefore be designed to work using mock data, manual entry, or file imports so that external integrations are not required for the core system to function.

Next Steps:

1. Create mock swimmer schedules and display them in the frontend.
2. Implement the first backend conflict-detection logic using the mock schedules.
3. Rank possible practice windows by number of conflicts and display the results to the coach.